Acroweb3 Portal Backend

Release v1.13.3

Alessandro Burastero

Jun 13, 2025

CONTENTS:

1 acroweb3_portal package **3**
- 1.1 acroweb3_portal.admin_app package . . . 3
- 1.2 acroweb3_portal.tests package . . . 32
- 1.3 acroweb3_portal.admin module . . . 33
- 1.4 acroweb3_portal.apps module . . . 34
- 1.5 acroweb3_portal.asgi module . . . 34
- 1.6 acroweb3_portal.auth module . . . 35
- 1.7 acroweb3_portal.db_router module . . . 39
- 1.8 acroweb3_portal.internal_views module . . . 41
- 1.9 acroweb3_portal.models module . . . 44
- 1.10 acroweb3_portal.prova module . . . 45
- 1.11 acroweb3_portal.serializers module . . . 45
- 1.12 acroweb3_portal.settings module . . . 46
- 1.13 acroweb3_portal.urls module . . . 47
- 1.14 acroweb3_portal.views module . . . 47
- 1.15 acroweb3_portal.websocket module . . . 54
- 1.16 acroweb3_portal.wsgi module . . . 58

2 activity_book package **59**
- 2.1 activity_book.admin module . . . 59
- 2.2 activity_book.apps module . . . 59
- 2.3 activity_book.manager module . . . 59
- 2.4 activity_book.models module . . . 61
- 2.5 activity_book.serializers module . . . 61
- 2.6 activity_book.views module . . . 61

3 manage module **65**

4 Indices and tables **67**
- 4.1 Github Repository . . . 67

Python Module Index **69**

Index **71**

CHAPTER

ONE

ACROWEB3_PORTAL PACKAGE

1.1 acroweb3_portal.admin_app package

1.1.1 acroweb3_portal.admin_app.idp_client module

Description:

This module provides the interface for the IDP client. The IDP client is the interface to the Identity Provider (IDP) that manages the users. The IDP client is responsible for the following tasks:

- get the list of users
- get the user data
- update the user data
- create a new user
- delete a user
- reset the user password
- create a domain
- delete a domain

Classes:

AdminUserData IDPClient

Functions:

getIDPClient(auth_data:AcrowebAuthData) -> IDPClient

return the IDP client for the given auth data

`acroweb3_portal.admin_app.idp_client.`**`idp_clients`**

dict

class `acroweb3_portal.admin_app.idp_client.`**`AdminUserData`**(*username: str = None, firstName: str = None, lastName: str = None, email: str = None, creationDate: datetime = None, phone: str = None, db_user:* UserData *= None*)

Bases: `object`

Description:

This class represents the user data for the admin interface. The user data is used to display the user information in the admin interface. The user data is also used to update the user information in the IDP.

username
str

firstName
str

lastName
str

email
str

creationDate
datetime

phone
str

db_user
UserData

to_dict() → dict
return the user data as a dictionary

set_from_db(user
UserData) set the user data from the database user data

create_db_user() → *UserData*
create a new database user data

update_db_user(user
UserData) update the database user data

set_from_props(props
dict) set the user data from the properties

to_dict()

set_from_db(*user:* UserData)

create_db_user()

update_db_user(*user:* UserData)

set_from_props(*props: dict*)

class acroweb3_portal.admin_app.idp_client.**IDPClient**(*auth_data: AcrowebAuthData*)

Bases: object

Description:
This class is the interface for the IDP client. The IDP client is the interface to the Identity Provider (IDP) that manages the users. The IDP client is responsible for the following tasks:

- create a new user
- delete a user
- reset the user password

auth_data

AcrowebAuthData

cfg

PortalConfig

issuer

str

user_exists(username

str) check if user exists

create_user(username

str, email:str, firstName: str, lastName: str, domain: str) create a new user

delete_user(username

str) delete an user

reset_user_password(username

str, new_password:str=None) rest user password

abstract user_exists(*username: str*)

check if user exists

abstract create_user(*username: str*, *email: str*, *firstName: str*, *lastName: str*, *domain: str*)

create a new user

abstract delete_user(*username: str*)

delete an user

abstract reset_user_password(*username: str*, *new_password: str = None*)

reset user password

acroweb3_portal.admin_app.idp_client.**getIDPClient**(*auth_data: AcrowebAuthData*) → *IDPClient*

Description:

return the IDP client for the given auth data

Parameters

auth_data – AcrowebAuthData the authentication data

Returns

IDPClient

the IDP client

Raises

None –

1.1.2 acroweb3_portal.admin_app.idp_db module

Description:
IDP client for the Acroweb3 portal backend that uses the database as the identity provider.

Classes:
DBIDPClient: IDP client for the Acroweb3 portal backend that uses the database as the identity provider.

`acroweb3_portal.admin_app.idp_db.`**`_logger`**
Logger for the module.

Type
Logger

Functions:
None

class `acroweb3_portal.admin_app.idp_db.`**`DBIDPClient`**(*auth_data: AcrowebAuthData*)
Bases: *IDPClient*

Description:
IDP client for the Acroweb3 portal backend that uses the database as the identity provider.

`__init__(self, auth_data`
AcrowebAuthData): Constructor for the DBIDPClient class.

`user_exists(self, username`
str) -> bool: Checks if a user exists.

`create_user(self, username`
str, email: str, firstName: str, lastName: str, domain: str): Creates a user.

`delete_user(self, username`
str): Deletes a user.

`reset_user_password(self, username`
str, new_password: str = None): Resets a user's password.

`user_exists`(*username: str*) → bool

Description:
Nothing to do. The user exists.. :).

Parameters
`username` (*str*) – The username.

Returns
True if the user exists, False otherwise.

Return type
bool

Raises
`None` –

`create_user`(*username: str*, *email: str*, *firstName: str*, *lastName: str*, *domain: str*)

Description:

Nothing to do.

Parameters

- **username** (*str*) – The username.
- **email** (*str*) – The email.
- **firstName** (*str*) – The first name.
- **lastName** (*str*) – The last name.
- **domain** (*str*) – The domain.

Returns
None

Raises
None –

delete_user(*username: str*)

Description:
Nothing to do.

Parameters
username (*str*) – The username.

Returns
None

Raises
None –

reset_user_password(*username: str*, *new_password: str = None*)

Description:
Nothing to do.

1.1.3 acroweb3_portal.admin_app.idp_keycloak module

Description:
This module implements the IDPClient interface for Keycloak. The KeycloakIDPClient class is a concrete implementation of the IDPClient interface. It uses the keycloak python client to interact with the Keycloak server.

Attributes:

- keycloak_cache: A dictionary that stores the KeycloakIDPClient instances for each issuer.
- _logger: A logger object for logging messages.

Functions:

- _keycloack_exception_handler: A decorator that catches KeycloakError exceptions and raises AcrowebException exceptions.

Classes:

- KeycloakIDPClient: A concrete implementation of the IDPClient interface for Keycloak.

class acroweb3_portal.admin_app.idp_keycloak.**KeycloakIDPClient**(***kwargs*)

Bases: *IDPClient*

Description:

A concrete implementation of the IDPClient interface for Keycloak. It uses the keycloak python client to interact with the Keycloak server.

- keycloak_admin

A KeycloakAdmin object that is used to interact with the Keycloak server.

- cache

A dictionary that stores the KeycloakIDPClient instance's cache.

Functions:

- __init__: Initializes the KeycloakIDPClient object.
- user_exists: Checks if a user exists.
- create_user: Creates a user.
- delete_user: Deletes a user.
- reset_user_password: Resets a user's password.

user_exists(***kwargs*)

check if user exists

create_user(***kwargs*)

create a new user

delete_user(***kwargs*)

delete an user

reset_user_password(***kwargs*)

reset user password

1.1.4 acroweb3_portal.admin_app.import_roles_in_db module

Description:

This script imports roles from keycloak to the database. It is used to import roles from keycloak to the database.

Functions:

get_db_domain(domain_name, portal) get_kc_user_id(keycloak_admin, username, portal) get_kc_client_id(keycloak_admin, client_name, portal) set_users_domains(keycloak_admin, portal) set_users_roles(keycloak_admin, portal) manage_portals() main

acroweb3_portal.admin_app.import_roles_in_db.**kc_user_id_cache**

dict

acroweb3_portal.admin_app.import_roles_in_db.**kc_client_id_cache**

dict

`acroweb3_portal.admin_app.import_roles_in_db.get_db_domain(domain_name, portal)`

Description:
This function gets the domain from the database.

Parameters

- **domain_name** (-) – str
- **portal** (-) – PortalConfig object

Returns
Domain object

Return type

- domain

Raises
- **Exception** – More than one domain with name {domain_name} found for portal {portal.id}

`acroweb3_portal.admin_app.import_roles_in_db.get_kc_user_id(keycloak_admin, username, portal)`

Description:
This function gets the user id from keycloak.

Parameters

- **keycloak_admin** (-) – KeycloakAdmin object
- **username** (-) – str
- **portal** (-) – PortalConfig object

Returns
str

Return type

- kc_user_id

Raises
- **Exception** – Error getting user_id: {e}

`acroweb3_portal.admin_app.import_roles_in_db.get_kc_client_id(keycloak_admin, client_name, portal)`

Description:
This function gets the client id from keycloak.

Parameters

- **keycloak_admin** (-) – KeycloakAdmin object
- **client_name** (-) – str
- **portal** (-) – PortalConfig object

Returns

str

Return type

- kc_client_id

Raises
- **Exception** – Error getting client_id: {e}

acroweb3_portal.admin_app.import_roles_in_db.**set_users_domains**(*keycloak_admin*, *portal*)

Description:
This function sets the users domains in the database.

Parameters

- **keycloak_admin** (-) – KeycloakAdmin object
- **portal** (-) – PortalConfig object

Returns

- None

Raises
- **None** –

acroweb3_portal.admin_app.import_roles_in_db.**set_users_roles**(*keycloak_admin*, *portal*)

Description:
This function sets the users roles in the database. If the role does not exist, it is created. Else, the role is added to the user. So the user has the role.

Parameters

- **keycloak_admin** (-) – KeycloakAdmin object
- **portal** (-) – PortalConfig object

Returns

- None

Raises
- **None** –

acroweb3_portal.admin_app.import_roles_in_db.**manage_portals**()

Description:
This function manages the portals. It gets the portals from the database. For each portal, it gets the keycloak admin. Then, it sets the users domains and roles.

Parameters
None (-)

Returns

- None

Raises
- **None** –

1.1.5 acroweb3_portal.admin_app.init_portal module

Description:
This script initializes a realm in keycloak and the corresponding domain in the database. It also creates the portal administrator and the machine user for the admin rest api. The script reads a json file containing the initialization data. The script creates the realm in keycloak, the domain in the database, the portal configuration, the portal administrator and the machine user for the admin rest api. The script also creates the roles for the applications and assigns the roles to the portal administrator. The script is intended to be used in the initialization of a new realm.

Classes:

- None

Functions:

- read_data_file(file_path:str) -> dict
- create_portal_config(data:dict, idp_url:str)
- create_user(data:dict, db_domain:Domain, keycloak_admin:KeycloakAdmin)
- create_domain(data:dict)
- create_applications(data_dict)

Attributes: - None

`acroweb3_portal.admin_app.init_portal.`**`read_data_file`**(*file_path: str*) → dict

Description:
This function reads a json file containing the initialization data.

Parameters
file_path (-) – str

Returns
dict

Return type

- data

Raises

- **- Exception** – invalid json file
- **- Exception** – invalid admin data in json file

`acroweb3_portal.admin_app.init_portal.`**`create_portal_config`**(*data: dict*, *idp_url: str*)

Description:
This function creates the portal configuration object in the database.

Parameters

- **data** (-) – dict
- **idp_url** (-) – str

Returns

- None

Raises
- **None** –

`acroweb3_portal.admin_app.init_portal.create_user(data: dict, db_domain: Domain, keycloak_admin: KeycloakAdmin)`

Description:
This function creates the portal administrator in keycloak and the database.

Parameters

- **data** (-) – dict
- **db_domain** (-) – Domain object
- **keycloak_admin** (-) – KeycloakAdmin object

Returns

- None

Raises
- **None** –

`acroweb3_portal.admin_app.init_portal.create_domain(data: dict)`

Description:
This function creates the domain in the database.

Parameters
data (-) – dict

Returns
Domain object

Return type

- db_domain

Raises
- **None** –

`acroweb3_portal.admin_app.init_portal.create_applications(data_dict)`

Description:
This function creates the applications in the database.

Parameters
data_dict (-) – dict

Returns

- None

Raises
- **None** –

1.1.6 acroweb3_portal.admin_app.internal_views module

Description:
This file contains the viewset for the community data. The community data viewset is used to get the community data from the IDP. The community data includes the users, domains and groups.

Classes:

- CommunityDataView: This class is a viewset for the community data.

class acroweb3_portal.admin_app.internal_views.**CommunityDataView**(***kwargs*)

Bases: ViewSet

Description:
This class is a viewset for the community data. The community data viewset is used to get the community data from the IDP. The community data includes the users, domains and groups.

Functions:

- list: This function is used to get the list of users from the IDP.
- domains: This function is used to get the list of domains from the IDP.
- groups: This function is used to get the list of groups from the IDP.
- user_domains: This function is used to get the list of domains of a user from the IDP.

list(*request*)

Description:
This function is used to get the list of users from the IDP.

Parameters
request (-) – Request object

Returns

- Response object

Raises
- **Response object** – If portal is not found in the request param.

domains(*request*)

Description:
This function is used to get the list of domains from the IDP.

Parameters
request (-) – Request object

Returns

- Response object

Raises
- **Response object** – If portal is not found in the request param.

groups(*request*)

Description:
This function is used to get the list of groups from the IDP.

Parameters
request (-) – Request object

Returns

- Response object

Raises
- **Response object** – If portal is not found in the request param.

user_domains(*request*)

Description:
This function is used to get the list of domains of a user from the IDP.

Parameters
request (-) – Request object

Returns

- Response object

Raises
- **Response object** – If portal or user is not found in the request param.

1.1.7 acroweb3_portal.admin_app.permissions module

Description:
This file contains the permissions for the admin app.

Functions:

- am_i_administrator: This function checks if the user is an administrator.
- am_i_coordinator: This function checks if the user is a coordinator.
- IsCoordinatorPermission: This class is a permission class for the coordinator.
- IsAdministratorPermission: This class is a permission class for the administrator.

acroweb3_portal.admin_app.permissions.**am_i_administrator**(*request*, *db_user=None*)

Description:
This function checks if the user is an administrator.

Parameters
request (-) – Request object

Returns

- bool

Raises
- **Exception** – Error getting user data: {e}

acroweb3_portal.admin_app.permissions.**am_i_cross_coordinator**(*request*, *db_user=None*)

Description:
This function checks if the user is a cross-coordinator.

Parameters
request (-) – Request object

Returns

- bool

Raises
- **Exception** – Error getting user data: {e}

acroweb3_portal.admin_app.permissions.**am_i_coordinator**(*request*, *db_user=None*)

Description:
This function checks if the user is a coordinator.

Parameters
request (-) – Request object

Returns

- bool

Raises
- **Exception** – Error getting user data: {e}

class acroweb3_portal.admin_app.permissions.**IsCoordinatorPermission**

Bases: IsAuthenticated

has_permission(*request*, *view*)

Description:
This function checks if the user is a coordinator.

Parameters

- **request** (-) – Request object

- **view** (-) – View object

Returns

- bool

Raises
- **None** –

class acroweb3_portal.admin_app.permissions.**IsAdministratorPermission**

Bases: IsAuthenticated

has_permission(*request*, *view*)

Description:
This function checks if the user is an administrator.

Parameters

- **request** (-) – Request object
- **view** (-) – View object

Returns

- bool

Raises
- **None** –

1.1.8 acroweb3_portal.admin_app.prv module

1.1.9 acroweb3_portal.admin_app.serializers module

Description:
This file contains the serializers for the admin app.

Classes:

- DomainSerializerForList: This class is a serializer for the domain list.
- ApplicationSerializerForWriter: This class is a serializer for the application writer.
- ApplicationSerializerForList: This class is a serializer for the application list.
- DomainSerializerForRetrieve: This class is a serializer for the domain retrieve.
- DomainSerializerForWriter: This class is a serializer for the domain writer.
- DomainGroupSerializer: This class is a serializer for the domain group.
- CommunityDomainSerializer: This class is a serializer for the community domain.
- CommunityDomainGroupSerializer: This class is a serializer for the community domain group.

Examples

```
>>> from acroweb3_portal.admin_app.serializers import DomainSerializerForList
>>> from acroweb3_portal.models import Domain
>>> domain = Domain.objects.get(id=1)
>>> serializer = DomainSerializerForList(domain)
>>> serializer.data
```

class acroweb3_portal.admin_app.serializers.**DomainSerializerForList**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

model

alias of *Domain*

fields = ['id', 'name', 'descr', 'imgUrl', 'color']

get_imgUrl(*obj*)

get_color(*obj*)

class acroweb3_portal.admin_app.serializers.**ApplicationSerializerForWriter**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

model

alias of *Application*

fields = ['id', 'category', 'text', 'title', 'icon', 'icon_svg', 'link', 'props']

create(*validated_data*)

We have a bit of extra checking around this in order to provide descriptive messages when something goes wrong, but this method is essentially just:

return ExampleModel.objects.create(**validated_data)

If there are many to many fields present on the instance then they cannot be set until the model is instantiated, in which case the implementation is like so:

example_relationship = validated_data.pop('example_relationship') instance = ExampleModel.objects.create(**validated_data) instance.example_relationship = example_relationship return instance

The default implementation also does not handle nested relationships. If you want to support writable nested relationships you'll need to write an explicit *.create()* method.

update(*instance*, *validated_data*)

delete(*instance*)

class acroweb3_portal.admin_app.serializers.**ApplicationSerializerForList**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

model

alias of *Application*

fields = ['id', 'text', 'title', 'category', 'icon', 'icon_url', 'icon_svg', 'props', 'link']

get_category(*obj*)

get_icon_url(*obj*)

Returns the icon as a URL-encoded SVG data URL

class acroweb3_portal.admin_app.serializers.**DomainSerializerForRetrieve**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

model

alias of *Domain*

fields = ['id', 'name', 'descr', 'lon1', 'lon2', 'lat1', 'lat2', 'date', 'props', 'imgUrl', 'color', 'apps']

get_imgUrl(*obj*)

get_color(*obj*)

class acroweb3_portal.admin_app.serializers.**DomainSerializerForWriter**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

model

alias of *Domain*

fields = ['id', 'name', 'descr', 'lon1', 'lon2', 'lat1', 'lat2', 'props']

create(*validated_data*)

We have a bit of extra checking around this in order to provide descriptive messages when something goes wrong, but this method is essentially just:

return ExampleModel.objects.create(**validated_data)

If there are many to many fields present on the instance then they cannot be set until the model is instantiated, in which case the implementation is like so:

example_relationship = validated_data.pop('example_relationship') instance = ExampleModel.objects.create(**validated_data) instance.example_relationship = example_relationship return instance

The default implementation also does not handle nested relationships. If you want to support writable nested relationships you'll need to write an explicit *.create()* method.

class acroweb3_portal.admin_app.serializers.**DomainGroupSerializer**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

model

alias of *DomainGroup*

fields = ['id', 'name', 'descr', 'color', 'date', 'imgUrl']

get_imgUrl(*obj*)

create(*validated_data*)

We have a bit of extra checking around this in order to provide descriptive messages when something goes wrong, but this method is essentially just:

return ExampleModel.objects.create(**validated_data)

If there are many to many fields present on the instance then they cannot be set until the model is instantiated, in which case the implementation is like so:

example_relationship = validated_data.pop('example_relationship') instance = ExampleModel.objects.create(**validated_data) instance.example_relationship = example_relationship return instance

The default implementation also does not handle nested relationships. If you want to support writable nested relationships you'll need to write an explicit *.create()* method.

class acroweb3_portal.admin_app.serializers.**CommunityDomainSerializer**(**args*, ***kwargs*)

Bases: *DomainSerializerForList*

class Meta

Bases: *Meta*

fields = ['id', 'name', 'descr', 'imgUrl', 'color', 'groups']

class acroweb3_portal.admin_app.serializers.**CommunityDomainGroupSerializer**(**args*, ***kwargs*)

Bases: *DomainGroupSerializer*

class Meta

Bases: *Meta*

fields = ['id', 'name', 'descr', 'color', 'date', 'imgUrl', 'domains']

1.1.10 acroweb3_portal.admin_app.views module

Description:

This file contains the viewset for the admin app. The admin app viewset is used to manage the users, roles, domains and applications.

Classes:

- AdminConfigView: This class is a viewset for the admin config.
- AdminUsersView: This class is a viewset for the admin users.
- AdminUploadView: This class is a viewset for the admin upload.
- AdminRolesView: This class is a viewset for the admin roles.
- AdminDomainsView: This class is a viewset for the admin domains.
- AdminApplicationsUpdateView: This class is a viewset for the admin applications update.

- AdminApplicationsView: This class is a viewset for the admin applications.
- AdminDomainGroupView: This class is a viewset for the admin domain group.

Functions:

- get_request_db_user_data: This function gets the user data from the request.
- get_domain_img_url: This function gets the domain image url.
- get_group_img_url: This function gets the group image url.
- get_user_img_url: This function gets the user image url.

class acroweb3_portal.admin_app.views.**AdminConfigView**(***kwargs*)

Bases: ViewSet

Description:
This class is a viewset for the admin config.

Functions:

- list: This function returns the configuration.

- permission_classes
List of permissions

permission_classes = [<class 'rest_framework.permissions.IsAuthenticated'>]

list(*request*)

class acroweb3_portal.admin_app.views.**AdminUsersView**(***kwargs*)

Bases: ViewSet

Description:
This class is a viewset for the admin users.

Functions:

- list: This function returns the list of users.
- me: This function returns the user data.
- user: This function returns the user data.
- save: This function saves the user data.
- delete: This function deletes the user data.

- permission_classes
List of permissions

permission_classes = [<class 'rest_framework.permissions.IsAuthenticated'>]

list(*request*)

Description:
This function returns the list of users.

Parameters

request (-) – Request object

Returns
Response object

Return type

- Response

Raises
- **None** –

me(*request*)

Description:
This function returns the user data.

Parameters
request (-) – Request object

Returns
Response object

Return type

- Response

Raises
- **None** –

user(*request*)

Description:
This function returns the user data.

Parameters
request (-) – Request object

Returns
Response object

Return type

- Response

Raises
- **None** –

save(*request*)

Description:
This function saves the user data. If the request method is post, it creates a new user. If the request method is put, it updates the user data.

Parameters
request (-) – Request object

Returns
Response object

Return type

- Response

Raises
- `None` –

`delete`(*request*)

Description:
This function deletes the user data.

Parameters
`request` (-) – Request object

Returns
Response object

Return type

- Response

Raises
- `None` –

`resetpassword`(*request*)

Description:
This function resets the user password.

Parameters
`request` (-) – Request object

Returns
Response object

Return type

- Response

Raises
- `None` –

`props`(*request*)

Description:

This function updates the user properties.
If the request method is patch, it updates the user properties. So the user can update his phoneNr, phoneEnabled, emailEnabled and color. Else it returns a response with the user properties.

Parameters
`request` (-) – Request object

Returns
Response object

Return type

- Response

Raises
- `None` –

`avatar`(*request*)

Description:
This function deletes the user avatar. If the request method is delete, it deletes the user avatar. Else it returns a response with the user avatar.

Parameters
`request` (-) – Request object

Returns
Response object

Return type

- Response

Raises
- `None` –

`roles`(*request*)

Description:
This function returns the user roles. If the request method is get, it returns the user roles. Else it returns a response with the user roles.

Parameters
`request` (-) – Request object

Returns
Response object

Return type

- Response

Raises
- `None` –

`role`(*request*)

Description:
This function updates the user roles. If the request method is put, it adds the user roles. If the request method is delete, it deletes the user roles. Else it returns a response with the user roles.

Parameters
`request` (-) – Request object

Returns
Response object

Return type

- Response

Raises
- `None` –

`domains`(*request*)

Description:
This function returns the user domains. If the request method is get, it returns the user domains. If the request method is put, it adds the user domains. If the request method is delete, it deletes the user domains. Else it returns a response with the user domains.

Parameters
`request` (-) – Request object

Returns
Response object

Return type

- Response

Raises
- `None` –

`class` acroweb3_portal.admin_app.views.`AdminUploadView`(***kwargs*)

Bases: `ViewSet`

Description:
This class is a viewset for the admin upload.

Functions:

- user: This function uploads the user image.
- domain: This function uploads the domain image.
- group: This function uploads the group image.
- __set_img: This function sets the image.

- `permission_classes`
List of permissions

- `parser_classes`
List of parsers

`permission_classes = [<class 'rest_framework.permissions.IsAuthenticated'>]`

`parser_classes = (<class 'rest_framework.parsers.MultiPartParser'>,)`

`user`(*request*, **args*, ***kwargs*)

Description:
This function uploads the user image.

Parameters
request (-) – Request object

Returns
Response object

Return type

- Response

Raises
- **None** –

domain(*request*, **args*, ***kwargs*)

Description:
This function uploads the domain image.

Parameters
request (-) – Request object

Returns
Response object

Return type

- Response

Raises
- **None** –

group(*request*, **args*, ***kwargs*)

Description:
This function uploads the group image. If the request method is post, it uploads the group image. Else it returns a response with the group image.

Parameters
request (-) – Request object

Returns
Response object

Return type

- Response

Raises
- **None** –

class acroweb3_portal.admin_app.views.**AdminRolesView**(***kwargs*)

Bases: ViewSet

Description:
This class is a viewset for the admin roles.

Functions:

- list: This function returns the list of roles.
- create: This function creates a new role.
- destroy: This function deletes a role.

`- permission_classes`
List of permissions

`permission_classes = [<class 'rest_framework.permissions.IsAuthenticated'>]`

`list`(*request*)

Description:
This function returns the list of roles. If the request method is get, it returns the list of roles. Else it returns a response with the list of roles.

Parameters
`request` (-) – Request object

Returns
Response object

Return type

- Response

Raises
`- None` –

`create`(*request*)

Description:
This function creates a new role. If the request method is post, it creates a new role. Else it returns a response with the list of roles.

Parameters
`request` (-) – Request object

Returns
Response object

Return type

- Response

Raises
`- None` –

`destroy`(*request*, *pk=None*)

Description:
This function deletes a role. If the request method is delete, it deletes a role.

Parameters

- **request** (-) – Request object
- **pk** (-) – Primary key

Returns
Response object

Return type

- Response

Raises
- **None** –

class acroweb3_portal.admin_app.views.**AdminDomainsView**(***kwargs*)

Bases: ModelViewSet

Description:
This class is a viewset for the admin domains.

Functions:

- get_queryset: This function returns the queryset.
- destroy: This function deletes a domain.
- get_serializer_class: This function returns the serializer class.

- permission_classes
List of permissions

permission_classes = [<class 'rest_framework.permissions.IsAuthenticated'>]

get_queryset()

Description:
This function returns the queryset. If the request user is an administrator, it returns the queryset of all domains. Else it returns the queryset of the domain.

Parameters
self (-) – Self object

Returns
QuerySet object

Return type

- QuerySet

Raises
- **None** –

destroy(*request*, **args*, ***kwargs*)

Description:
This function deletes a domain. If the request method is delete, it deletes a domain.

Parameters
request (-) – Request object

Returns
Response object

Return type

- Response

Raises
- **None** –

get_serializer_class()

Description:
This function returns the serializer class.

Parameters
self (-) – Self object

Returns
Serializer object

Return type

- Serializer

Raises
- **None** –

image(*request*, *domain_id*)

Description:
Clears the image associated with a domain.

Parameters

- **request** (*HttpRequest*) – The HTTP request object.
- **domain_id** (*int*) – The ID of the domain.

Returns
An empty response.

Return type
Response

app(*request*, *domain_id*)

Description:
Adds or removes an app from a domain. If the request method is put, it adds the app to the domain. If the request method is delete, it removes the app from the domain.

Parameters

- **request** (-) – Request object
- **domain_id** (-) – Domain ID

Returns
Response object

Return type

- Response

Raises
- **None** –

coordinator(*request*)

Description:
Adds or removes a coordinator from a domain. If the request method is put, it adds the coordinator to the domain. If the request method is delete, it removes the coordinator from the domain.

Parameters
request (-) – Request object

Returns
Response object

Return type

- Response

Raises
- **None** –

users(*request*, *domain_id*)

Description:
This function returns the users of a domain.

Parameters

- **request** (-) – Request object
- **domain_id** (-) – Domain ID

Returns
Response object

Return type

- Response

Raises
- **None** –

class acroweb3_portal.admin_app.views.**AdminApplicationsUpdateView**(***kwargs*)

Bases: CreateModelMixin, DestroyModelMixin, UpdateModelMixin, GenericViewSet

Description:
This class is a viewset for the admin applications update.

Functions:

- get_queryset: This function returns the queryset.
- get_serializer_class: This function returns the serializer class.

- permission_classes
List of permissions

permission_classes = [<class 'acroweb3_portal.admin_app.permissions.IsAdministratorPermission'>]

queryset

serializer_class
alias of *ApplicationSerializerForWriter*

class acroweb3_portal.admin_app.views.**AdminApplicationsView**(***kwargs*)
Bases: ListModelMixin, GenericViewSet

Description:
This class is a viewset for the admin applications.

Functions:

- get_queryset: This function returns the queryset.
- get_serializer_class: This function returns the serializer class.

- permission_classes
List of permissions

permission_classes = [<class 'acroweb3_portal.admin_app.permissions.IsCoordinatorPermission'>]

queryset

serializer_class
alias of *ApplicationSerializerForList*

class acroweb3_portal.admin_app.views.**AdminDomainGroupView**(***kwargs*)
Bases: ModelViewSet

Description:
This class is a viewset for the admin domain group.

Functions:

- get_queryset: This function returns the queryset.
- domains: This function returns the domains.
- image: This function deletes the image.
- domain: This function adds or removes a domain.

- permission_classes
List of permissions

- `serializer_class`
  Serializer class

`permission_classes = [<class 'acroweb3_portal.admin_app.permissions.IsAdministratorPermission'>]`

`serializer_class`
alias of *DomainGroupSerializer*

`get_queryset()`

Description:
This function returns the queryset. If the request user is an administrator, it returns the queryset of all domain groups. Else it returns the queryset of the domain group.

Parameters
`self` (-) – Self object

Returns
QuerySet object

Return type

- QuerySet

Raises
- `None` –

`domains(request, group_id)`

Description:
This function returns the domains of a domain group.

Parameters

- `request` (-) – Request object
- `group_id` (-) – Group ID

Returns
Response object

Return type

- Response

Raises
- `None` –

`image(request, group_id)`

Description:
This function deletes the image associated with a domain group.

Parameters

- `request` (-) – Request object

- **group_id** (-) – Group ID

Returns
Response object

Return type

- Response

Raises
- **None** –

domain(*request*, *group_id*)

Description:
This function adds or removes a domain from a domain group. If the request method is put, it adds the domain to the domain group. If the request method is delete, it removes the domain from the domain group.

Parameters

- **request** (-) – Request object
- **group_id** (-) – Group ID

Returns
Response object

Return type

- Response

Raises
- **None** –

1.1.11 Module contents

acroweb3_portal.admin_app.**get_domain_img_url**(*obj:* Domain) → str

acroweb3_portal.admin_app.**get_group_img_url**(*obj:* DomainGroup) → str

acroweb3_portal.admin_app.**get_user_img_url**(*obj:* UserData) → str

acroweb3_portal.admin_app.**get_request_db_user_data**(*request*)

1.2 acroweb3_portal.tests package

1.2.1 acroweb3_portal.tests.commons module

acroweb3_portal.tests.commons.**init_db**()

1.2.2 acroweb3_portal.tests.dds_clients module

Created on Apr 14, 2020

@author: doy

1.2.3 acroweb3_portal.tests.sentinel_client module

Created on Apr 14, 2020 @author: doy

Description:
This file contains the viewset for the community data. The community data viewset is used to get the community data from the IDP. The community data includes the users, domains and groups.

1.2.4 acroweb3_portal.tests.test_views module

1.3 acroweb3_portal.admin module

Description:
This file contains the admin configuration for the acroweb3_portal app.

Classes:

- Acroweb3Admin: This class is a base class for the admin configuration.
- ApplicationAdmin: This class is an admin configuration for the Application model.
- UserDataAdmin: This class is an admin configuration for the UserData model.

class acroweb3_portal.admin.**Acroweb3Admin**(*model*, *admin_site*)

Bases: `ModelAdmin`

save_as = True

list_max_show_all = 1000

list_per_page = 200

search_fields = ['name']

formfield_overrides = {<class 'django.db.models.fields.json.JSONField'>: {'widget': <class 'django_json_widget.widgets.JSONEditorWidget'>}}

property media

class acroweb3_portal.admin.**ApplicationAdmin**(*model*, *admin_site*)

Bases: *Acroweb3Admin*

search_fields = ['id']

list_filter = ('category',)

list_display = ('icon_svg_view', 'id', 'title', 'category', 'link')

list_display_links = ('icon_svg_view', 'id', 'title')

readonly_fields = ['icon_svg_view']

icon_svg_view(*obj*)

property media

class acroweb3_portal.admin.**UserDataAdmin**(*model*, *admin_site*)

Bases: *Acroweb3Admin*

list_display = ('username', 'email', 'first_name', 'last_name', 'domain')

list_display_links = ('username', 'email', 'first_name', 'last_name')

list_filter = ('domain',)

filter_horizontal = ('roles', 'extra_domains')

readonly_fields = ['img_view']

search_fields = ['username', 'email', 'first_name', 'last_name']

img_view(*obj*)

property media

1.4 acroweb3_portal.apps module

class acroweb3_portal.apps.**Acroweb3Config**(*app_name*, *app_module*)

Bases: AppConfig

default_auto_field = 'django.db.models.BigAutoField'

1.5 acroweb3_portal.asgi module

Description:

ASGI config for cbe project. If settings.ACTIVATE_WS is True, it will also activate the websocket. Else it will not activate the websocket. The websocket is used for the notification system.

- application

ProtocolTypeRouter object

- router_data

dict

- WSConsumer

WSConsumer object

- WSTokenAuthMiddleware

WSTokenAuthMiddleware object

- WSNotifier

WSNotifier object

1.6 acroweb3_portal.auth module

Description:

This file contains the authentication component for the portal.

Classes:

- AuthenticationComponent: This class is an abstract class for the authentication component.
- AuthDomainSerializer: This class is a serializer for the domain.
- PortalAuthentication: This class is an authentication component for the portal.
- PortalAuthenticationReverseProxy: This class is an authentication component for the portal with reverse proxy.

- _logger

Logger object

class acroweb3_portal.auth.**AuthDomainSerializer**(**args*, ***kwargs*)

Bases: `ModelSerializer`

class Meta

Bases: `object`

model

alias of *Domain*

fields = '__all__'

class acroweb3_portal.auth.**AuthenticationComponent**

Bases: `object`

Description:

This class is an abstract class for the authentication component.

Functions:

- _build_auth_data: This function builds the authentication data.
- validate_token: This function validates the token.
- validate_token_and_build_auth_data: This function validates the token and builds the authentication data.

abstract validate_token(*issuer*, *client_id=None*)

Description:

This function validates the token.

Parameters

- **access_token** (-) – Access token
- **issuer** (-) – Issuer

- **client_id** (-) – Client ID

Returns
Token information

Return type

- token_info

abstract validate_token_and_build_auth_data(*access_token*, *host*, *client_id*, *chosen_domain=None*) → AcrowebAuthData

Description:
This function validates the token and builds the authentication data.

Parameters

- **access_token** (-) – Access token
- **host** (-) – Host
- **client_id** (-) – Client ID
- **chosen_domain** (-) – Chosen domain

Returns
AcrowebAuthData object

Return type

- auth_data

Raises

- **- AcrowebException** – cannot find config for host {host}
- **- AcrowebException** – found more tha one configuration ({len(configurations)}) for host {host}

acroweb3_portal.auth.**get_auth_component**() → *AuthenticationComponent*

Description:
This function gets the authentication component.

Returns
AuthenticationComponent object

Return type

- auth_component

Raises
- AcrowebException – Authentication component not defined

acroweb3_portal.auth.**get_portal_config_by_host**(*host*) → *PortalConfig*

Description:
This function is used to retrive the portal configuration for a particular host.

Parameters
host (-) – str

Returns
PortalConfig object

Return type

- configurations[0]

`acroweb3_portal.auth.get_portal_and_issuer_by_host(host)` → Tuple[str, str]

Description:
This function gets the portal and issuer by host.

Parameters
host (-) – str

Returns
str - issuer: str

Return type

- cfg.id

class `acroweb3_portal.auth.PortalAuthentication`

Bases: `AcrowebOAuth2Authentication`, *AuthenticationComponent*

Description:
This class is an authentication component for the portal.

Functions:

- validate_token: This function validates the token.
- validate_token_and_build_auth_data: This function validates the token and builds the authentication data.

`validate_token(access_token, issuer, client_id=None)`

Description:
This function validates the token.

Parameters

- **access_token** (-) – Access token
- **issuer** (-) – Issuer
- **client_id** (-) – Client ID

Returns
Token information

Return type

- token_info

Raises

- **AcrowebException** – {str(e)}

validate_token_and_build_auth_data(*access_token*, *host*, *client_id*, *chosen_domain=None*) → AcrowebAuthData

Description:
This function validates the token and builds the authentication data.

Parameters

- **access_token** (-) – Access token
- **host** (-) – Host
- **client_id** (-) – Client ID
- **chosen_domain** (-) – Chosen domain

Returns
AcrowebAuthData object

Return type

- auth_data

Raises

- - **AcrowebException** – {str(iae)}
- - **AcrowebException** – {str(e)}

class acroweb3_portal.auth.**PortalAuthenticationReverseProxy**

Bases: AcrowebOAuth2Authentication, *AuthenticationComponent*

Description:
This class is an authentication component for the portal with reverse proxy.

Functions:

- validate_token: This function validates the token.
- validate_token_and_build_auth_data: This function validates the token and builds the authentication data.

validate_token(*access_token*, *issuer*, *client_id=None*)

Description:
This function validates the token.

Parameters

- **access_token** (-) – Access token
- **issuer** (-) – Issuer
- **client_id** (-) – Client ID

Returns
Token information

Return type

- token_info

Raises
- **AcrowebException** – {str(e)}

validate_token_and_build_auth_data(*access_token*, *host*, *client_id*, *chosen_domain=None*) → AcrowebAuthData

Description:
This function validates the token and build auth data.

Parameters

- **access_token** (-) – Access token
- **host** (-) – Host
- **client_id** (-) – Client ID
- **chosen_domain** (-) – Chosen domain

Return type

- auth_data

Raises

- - **AcrowebException** – {str(iae)}
- - **AcrowebException** – {str(e)}

authenticate(*request*)

Description:
This function authenticates the user.

Parameters
request (-) – Request object

Returns
User object - auth: Auth object

Return type

- user

1.7 acroweb3_portal.db_router module

Description:
This file contains the database router for the activity_book app.

Classes:

- ActivityRouter: This class is a database router for the activity_book app.

class acroweb3_portal.db_router.**ActivityRouter**
Bases: object

Description:
This class is a database router for the activity_book app.

Functions:

- db_for_read: This function is used to get the database for read.
- db_for_write: This function is used to get the database for write.
- allow_relation: This function is used to allow relation.
- allow_migrate: This function is used to allow migration.

`- route_app_labels`
list

`route_app_labels = {'activity_book'}`

`db_for_read`(*model*, ***hints*)

Description:
This function is used to get the database for read.

Parameters

- `model` (-) – Model object
- `hints` (-) – dict

Returns

- str

`db_for_write`(*model*, ***hints*)

Description:
This function is used to get the database for write.

Parameters

- `model` (-) – Model object
- `hints` (-) – dict

Returns

- str

`allow_relation`(*obj1*, *obj2*, ***hints*)

Description:
This function is used to allow relation.

Parameters

- `obj1` (-) – Model object
- `obj2` (-) – Model object

- **hints** (-) – dict

Returns

- bool

allow_migrate(*db*, *app_label*, *model_name=None*, ***hints*)

Description:
This function is used to allow migration.

Parameters

- **db** (-) – str
- **app_label** (-) – str
- **model_name** (-) – str
- **hints** (-) – dict

Returns

- bool

Raises
- **None** –

1.8 acroweb3_portal.internal_views module

Created on Oct 22, 2019

@author: doy

Description:
This file contains the internal views for the portal. The internal views are used to manage the portal data.

Classes:

- IssuerView: This class is a viewset for the issuer.
- AuthenticateView: This class is a viewset for the authentication.
- InternalDomainView: This class is a viewset for the internal domain.
- WSPushView: This class is a viewset for the web socket push.
- InternalActivityWriterView: This class is a viewset for the internal activity writer.

- _logger
Logger object

class acroweb3_portal.internal_views.**IssuerView**(***kwargs*)
Bases: ViewSet

Description:
This class is a viewset for the issuer.

Functions:

- list: This function is used to get the issuer.

Raises
- **Response object** – If portal is not found in the request param.

Returns

- Response object

list(*request*)

class acroweb3_portal.internal_views.**AuthenticateView**(***kwargs*)

Bases: ViewSet

Description:
This class is a viewset for the authentication.

Functions:

- create: This function is used to create the authentication.

Raises
- **Response object** – If the body is invalid.

Returns

- Response object

create(*request*)

class acroweb3_portal.internal_views.**InternalDomainView**(***kwargs*)

Bases: GenericViewSet, RetrieveModelMixin

Description:
This class is a viewset for the internal domain.

queryset
Queryset object

serializer_class
Serializer

Returns

- Response object

queryset

serializer_class
alias of *DomainSerializer*

class acroweb3_portal.internal_views.**WSPushView**(***kwargs*)

Bases: ViewSet

Description:
This class is a viewset for the web socket push.

Functions:

- create: This function is used to create the web socket push.

Raises
- **Response object** – If the body is invalid.

Returns

- Response object

create(*request*)

class acroweb3_portal.internal_views.**InternalActivityWriterView**(***kwargs*)

Bases: ViewSet

Description:
This class is a viewset for the internal activity writer.

Functions:

- open_session: This function is used to open/close a session.
- close_session: This function is used to open/close a session.
- create: This function is used to create the activity.

open_session(*request*)

Description:
This function is used to open a session.

Parameters
request (-) – Request object

Returns

- Response object

Raises
- **Response object** – If the body is invalid.

close_session(*request*)

Description:
This function is used to close a session.

Parameters
request (-) – Request object

Returns

- Response object

Raises
- **Response object** – If the query params "user" and "portal" are required.

create(*request*)

Description:
This function is used to create an activity. Try to create an activity and return the id.

Parameters
request (-) – Request object

Returns

- Response object

Raises
- **Response object** – If the body is invalid.

1.9 acroweb3_portal.models module

Description:
This file contains the models for the portal app.

Classes:

- ApplicationCategory: This class is a model for the application category.
- Application: This class is a model for the application.
- ApplicationRole: This class is a model for the application role.
- PortalConfig: This class is a model for the portal config.
- DomainGroup: This class is a model for the domain group.
- Domain: This class is a model for the domain.
- UserData: This class is a model for the user data.
- DashboardSettings: This class is a model for the dashboard settings.
- WidgetType: This class is a model for the widget type.
- Widget: This class is a model for the widget.
- WidgetInstance: This class is a model for the widget instance.
- DomainWidgetSettings: This class is a model for the domain widget settings.
- UserWidgetSettings: This class is a model for the user widget settings.

class acroweb3_portal.models.**ApplicationCategory**(*id*, *category*, *title*, *order*)

class acroweb3_portal.models.**Application**(*id*, *category*, *text*, *title*, *always_visible*, *icon*, *icon_svg*, *link*, *props*)

class acroweb3_portal.models.**ApplicationRole**(*id*, *application*, *name*, *descr*)

class acroweb3_portal.models.**PortalConfig**(*id*, *hosts*, *auth*, *skin*, *partner_svg*, *owner_svg*, *desktop_logo_svg*, *mobile_logo_svg*, *favicon_svg*, *background_svg*, *apps_config*, *credits*)

class acroweb3_portal.models.**DomainGroup**(*id*, *name*, *portal*, *descr*, *date*, *img*, *color*)

class acroweb3_portal.models.**Domain**(*id*, *name*, *portal*, *descr*, *lon1*, *lon2*, *lat1*, *lat2*, *date*, *props*, *img*)

class acroweb3_portal.models.**UserData**(*id*, *username*, *first_name*, *last_name*, *email*, *email_enabled*, *creation_date*, *phone_nr*, *phone_enabled*, *color*, *domain*, *portal*, *props*, *img*)

validate_unique(*exclude: Collection[str] | None = Ellipsis*) → None

Check unique constraints on the model and raise ValidationError if any failed.

save(**args*, ***kwargs*) → None

Save the current instance. Override this in a subclass if you want to control the saving process.

The 'force_insert' and 'force_update' parameters can be used to insist that the "save" must be an SQL insert or update (or equivalent for non-SQL backends), respectively. Normally, they should not be set.

class acroweb3_portal.models.**DashboardSettings**(*id*, *portal*, *domain*, *user*, *settings*)

class acroweb3_portal.models.**WidgetType**(*name*, *descr*)

class acroweb3_portal.models.**Widget**(*name*, *title*, *widget_type*, *data_url*, *settings*)

class acroweb3_portal.models.**WidgetInstance**(*id*, *widget*, *portal*, *settings*)

class acroweb3_portal.models.**DomainWidgetSettings**(*id*, *domain*, *widget_instance*, *settings*)

class acroweb3_portal.models.**UserWidgetSettings**(*id*, *user*, *widget_instance*, *settings*)

class acroweb3_portal.models.**UserAppSettings**(*id*, *user*, *app*, *key*, *data*)

1.10 acroweb3_portal.prova module

1.11 acroweb3_portal.serializers module

Description:

This file contains the serializers for the admin app.

Classes:

- ApplicationSerializer: This class is a serializer for the application.
- ApplicationCategorySerializer: This class is a serializer for the application category.
- DomainSerializer: This class is a serializer for the domain.

Functions:

- get_enabled: This function gets the enabled status.
- get_roles: This function gets the roles.
- get_icon_url: This function gets the icon URL.

- get_request_db_user_data: This function gets the request database user data.

class acroweb3_portal.serializers.**ApplicationSerializer**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

model

alias of *Application*

fields = ('id', 'category', 'text', 'title', 'icon', 'icon_url', 'icon_svg', 'link', 'props', 'enabled', 'roles')

get_enabled(*obj*)

get_roles(*obj*)

get_icon_url(*obj*)

Returns the icon as a URL-encoded SVG data URL

class acroweb3_portal.serializers.**ApplicationCategorySerializer**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

model

alias of *ApplicationCategory*

fields = '__all__'

class acroweb3_portal.serializers.**DomainSerializer**(**args*, ***kwargs*)

Bases: ModelSerializer

class Meta

Bases: object

model

alias of *Domain*

fields = '__all__'

1.12 acroweb3_portal.settings module

Django settings for acroweb3 project.

Generated by 'django-admin startproject' using Django 3.1.3.

For more information on this file, see https://docs.djangoproject.com/en/3.1/topics/settings/

For the full list of settings and their values, see https://docs.djangoproject.com/en/3.1/ref/settings/

1.13 acroweb3_portal.urls module

acroweb3 URL Configuration

The *urlpatterns* list routes URLs to views. For more information please see:
https://docs.djangoproject.com/en/3.1/topics/http/urls/

Examples: Function views

1. Add an import: from my_app import views
2. Add a URL to urlpatterns: path(‘’, views.home, name=’home’)

Class-based views

1. Add an import: from other_app.views import Home
2. Add a URL to urlpatterns: path(‘’, Home.as_view(), name=’home’)

Including another URLconf

1. Import the include() function: from django.urls import include, path
2. Add a URL to urlpatterns: path(‘blog/’, include(‘blog.urls’))

1.14 acroweb3_portal.views module

Created on Oct 22, 2019 @author: doy

Description:
This file contains the views for the portal. The views are used to get the portal data.

Classes:

- ServerTimeView: This class is a viewset for the server time.
- UserImagesView: This class is a viewset for the user images.
- ApplicationCategoryView: This class is a viewset for the application category.
- ImageView: This class is a viewset for the image.
- PortalConfigView: This class is a viewset for the portal config.
- DashboardSettingsView: This class is a viewset for the dashboard settings.
- WidgetView: This class is a viewset for the widget.
- TranslationView: This class is a viewset for the translation.

Functions:

- get_host_by_request: This function is used to get the host by request.
- get_logger: This function is used to get the logger.
- get_portal_config_by_host: This function is used to get the portal config by host.

`class acroweb3_portal.views.ServerTimeView(**kwargs)`

Bases: `ViewSet`

Description:
server time resource

Functions:
list: This function is used to get the server time.

list(*request*)
server time resource

class acroweb3_portal.views.**UserImagesView**(***kwargs*)

Bases: `ViewSet`

Description:
user images resource

permission_classes
List of permission classes

Functions:
_domain_data_from_db_domain: This function is used to get the domain data from the database domain. list: This function is used to get the user images.

permission_classes = [<class 'rest_framework.permissions.IsAuthenticated'>]

list(*request*)

Description:
This function is used to get the user images.

- **auth_data**
  AcrowebAuthData object
- **user**
  Database user object
- **domains**
  List of database domain objects
- **allowed_domains**
  List of allowed domains
- **current_domain**
  Current domain
- **data**
  dict

Parameters
request (-) – Request object

Returns

- Response object

class acroweb3_portal.views.**ApplicationCategoryView**(***kwargs*)

Bases: ReadOnlyModelViewSet

Description:
application category resource

permission_classes
List of permission classes

Functions:
get_queryset: This function is used to get the queryset.

serializer_class
alias of *ApplicationCategorySerializer*

get_queryset()

Description:
This function is used to get the queryset.

- **auth_data**
  AcrowebAuthData object
- **domains**
  List of database domain objects
- **apps_id**
  List of application ids

Returns

- Queryset

Raises
- **Response object** – If domain is not found in the database.

class acroweb3_portal.views.**ImageView**(***kwargs*)

Bases: ViewSet

Description:
image resource

permission_classes
List of permission classes

Functions:

- user: This function is used to get the user image.
- domain: This function is used to get the domain image.
- group: This function is used to get the group image.

permission_classes = [<class 'rest_framework.permissions.IsAuthenticated'>]

user(*request*, **args*, ***kwargs*)

Description:
This function is used to get the user image.

- **auth_data**
  AcrowebAuthData object
- **username**
  Username
- **users**
  List of database user objects
- **user**
  Database user object

Return

- HttpResponse object

Raises
- **Response object** – If user is not found in the database.

domain(*request*, **args*, ***kwargs*)

Description:
This function is used to get the domain image.

- **auth_data**
  AcrowebAuthData object
- **domain_id**
  Domain id
- **domain**
  Database domain object

Parameters
request (-) – Request object

Returns

- HttpResponse object

Raises
- **Response object** – If domain is not found in the database.

group(*request*, **args*, ***kwargs*)

Description:
This function is used to get the group image.

- **group_id**
  Group id

Parameters
request (-) – Request object

Returns

- HttpResponse object

class acroweb3_portal.views.**PortalConfigView**(***kwargs*)

Bases: ViewSet

Description:
portal config resource

Functions:
list: This function is used to get the portal config.

list(*request*)

Description:
portal config resource

Parameters
request (-) – Request object

Returns

- Response object

Raises
- **Response object** – If an error occurs.

class acroweb3_portal.views.**DashboardSettingsView**(***kwargs*)

Bases: ViewSet

Description:
dashboard settings resource

Functions:
list: This function is used to get the dashboard settings.

permission_classes
List of permission classes

permission_classes = [<class 'rest_framework.permissions.IsAuthenticated'>]

list(*request*)

Description:
dashboard settings resource

Parameters
request (-) – Request object

Returns

- Response object

Raises

- **Response object** – If an error occurs.

class acroweb3_portal.views.**WidgetView**(***kwargs*)

Bases: ViewSet

Description:
widget resource

permission_classes
List of permission classes

Functions:
list: This function is used to get all allowed widgets with specific settings for the user. save_widget_settings: This function is used to save user settings for a widget.

permission_classes = [<class 'rest_framework.permissions.IsAuthenticated'>]

list(*request*)

Description:
return all allowed widgets with specific settings for the user

Parameters
request (-) – Request object

Returns

- Response object

Raises
- **Response object** – If an error occurs.

save_widget_settings(*request*, *widget_instance_id*)

Description:
save user settings for a widget

Parameters

- **request** (-) – Request object
- **widget_instance_id** (-) – Widget instance id

Returns

- Response object

Raises
- **Response object** – If an error occurs.

mydomains(*request*, *widget_instance_id*)

Description:
return all allowed domains for a widget

Parameters

- **request** (-) – Request object
- **widget_instance_id** (-) – Widget instance id

Returns

- Response object

Raises
- **Response object** – If an error occurs.

allow_domain(*request*, *widget_instance_id*, *domain_id*)

Description:
disallow domain for a widget

Parameters

- **request** (-) – Request object
- **widget_instance_id** (-) – Widget instance id
- **domain_id** (-) – Domain id

Returns

- Response object

Raises
- **Response object** – If an error occurs.

class acroweb3_portal.views.**TranslationView**(***kwargs*)

Bases: GenericViewSet

Description:
Translation view

Functions:
translate: This function is used to return translation json

translate(*request*, *portal*, *app*, *language*)

Description:
Return translation json

Parameters

- **request** (-) – Request object
- **portal** (-) – Portal name
- **app** (-) – Application name
- **language** (-) – Language name

Returns

- Response object

Raises
- **Exception** – If an error occurs.

class acroweb3_portal.views.**UserAppSettingsView**(***kwargs*)

Bases: ViewSet

Description:
user app settings resource

permission_classes
List of permission classes

Functions:
list: This function is used to get the user app settings.

permission_classes = [<class 'rest_framework.permissions.IsAuthenticated'>]

get_settings(*request*, *app_id*)

1.15 acroweb3_portal.websocket module

Created on Feb 5, 2020

@author: doy

Description:
websocket consumer for acroweb3 portal

This module is responsible for handling websocket connections and notifications. It is used by the acroweb3-portal-backend to notify web clients of events. It is also used by the acroweb3-portal-backend to notify web clients of events.

Classes:
WSTokenAuthMiddleware: Custom token auth middleware WSConsumer: Notifier for web clients WSNotifier: CBC listening class

Functions:

- _get_portal_config_by_host: Get portal config by host

- _logger
Logger object

class acroweb3_portal.websocket.**WSTokenAuthMiddleware**(*inner*)

Bases: object

Description:
Custom token auth middleware

Functions:
__init__: Constructor __call__: Call function

class acroweb3_portal.websocket.**WSConsumer**(**args*, ***kwargs*)

Bases: JsonWebsocketConsumer

Description:
Notifier for web clients

Functions:
connect: Connect function match: Match function disconnect: Disconnect function notify: Notify function notify_portal: Notify portal function

`notifier_listener`
None

Type
list

`notifier_listener: list = [<acroweb3_portal.websocket.WSNotifier object>]`

`connect()`

Description:
Connect function, used to connect to the websocket It logs the connection request If the user is not found, it logs an error and returns the accept function If the notifier listener is None, it logs a message and creates a new CmdNotifierlistener It adds the notifier to the notifier listener It accepts the connection

Parameters
`self` – Self

Returns
self.accept() self.send(text_data=msg, close=4001)

Raises
`Exception` – If the user is not found, it logs an error and returns the accept function

`match(portal, path)`

Description:
Match function, used to match the portal and path It gets the path filter from the scope If the path filter is None or empty or the portal is None or empty, it returns False else it returns the result of the fnmatch function

Parameters

- `portal` – Portal
- `path` – Path

Returns
fnmatch function

`disconnect(close_code)`

Description:
Disconnect function, used to disconnect from the websocket It logs the disconnect request If the notifier listener is None, it logs a message and creates a new CmdNotifierlistener It removes the notifier from the notifier listener

Parameters

- `self` – Self
- `close_code` – Close code

Returns
None

Raises
Exception – If the notifier listener is None, it logs a message and creates a new CmdNotifierlistener

static notify(*portal: str*, *path: str*, *obj: dict*) → int

Description:
Notify function, used to notify the portal, path and object If the settings activate ws is True, it logs a message and creates a new CmdNotifierlistener It returns the result of the notify function

Parameters

- **portal** – Portal
- **path** – Path
- **obj** – Object

Returns
notify function

static notify_portal(*portal: str*, *text: str*) → int

Description:
Notify portal function, used to notify the portal and text It returns the result of the notify function

Parameters

- **portal** – Portal
- **text** – Text

Returns
notify function

Raises
Exception – If the portal is not found, it logs an error and returns the notify function

class acroweb3_portal.websocket.**WSNotifier**

Bases: object

Description:
base web socket notifier class

Functions:
__init__: Constructor add_notifier: Add notifier function remove_notifier: Remove notifier function on_message: On message function notify: Notify function

__notifiers
None

add_notifier(*notifier:* WSConsumer)

Description:
Add notifier function, used to add a notifier It appends the notifier to the list of notifiers

Parameters
notifier – WSConsumer

Returns
None

Raises
Exception – If the notifier is not found, it logs an error and returns the append function

remove_notifier(*notifier: WSConsumer*)

Description:
Remove notifier function, used to remove a notifier It removes the notifier from the list of notifiers

Parameters
notifier – WSConsumer

Returns
None

Raises
Exception – If the notifier is not found, it logs an error and returns the remove function

notify(*portal: str*, *path: str*, *obj: dict*) → int

Description:
Notify function, used to notify the portal, path and object It logs a message and initializes the notified count It iterates over the notifiers If the notifier matches the portal and path, it sends the json object It increments the notified count It logs a message and returns the notified count

Parameters

- **portal** (-) – Portal
- **path** (-) – Path
- **obj** (-) – Object

Returns
notified_count

Raises
Exception – If the notifier matches the portal and path, it sends the json

class acroweb3_portal.websocket.**KafkaWSNotifier**

Bases: *WSNotifier*, BrokerSubscriber

Description:
web socket notifier class as kakfa broker subscriber

Functions:
__init__: Constructor on_message: On message function

on_message(*obj: object*, *body: str*, *routing_key: str*) → None

Description:
On message function, used to notify the offset, object and body It logs a message and parses the notification body

If the path is not found, it logs a warning and returns None If the object is not found, it logs a warning and returns None It notifies the portal, path and body

Parameters

- **offset** (-) – Offset
- **obj** (-) – Object
- **body** (-) – Body

Returns
None

Raises
Exception – If the path is not found, it logs a warning and returns None

class acroweb3_portal.websocket.**SentinelWSNotifier**

Bases: *WSNotifier*, SentinelListener

Description:
web socket notifier class as rabbitmq broker subscriber

Functions:
__init__: Constructor on_message: On message function

callback(*data: dict*, *routing_key: str*)

Callback method to be implemented by the listener

class acroweb3_portal.websocket.**NewAcqWSNotifier**

Bases: *WSNotifier*, NewAcqListener

Description:
web socket notifier class as rabbitmq broker subscriber

Functions:
__init__: Constructor on_message: On message function

callback(*data: dict*, *routing_key: str*)

Callback method to be implemented by the listener

1.16 acroweb3_portal.wsgi module

WSGI config for cbe project.

It exposes the WSGI callable as a module-level variable named `application`.

For more information on this file, see https://docs.djangoproject.com/en/2.2/howto/deployment/wsgi/

CHAPTER

TWO

ACTIVITY_BOOK PACKAGE

2.1 activity_book.admin module

2.2 activity_book.apps module

class activity_book.apps.**ActivityBookConfig**(*app_name*, *app_module*)

Bases: AppConfig

default_auto_field = 'django.db.models.BigAutoField'

2.3 activity_book.manager module

Description:

This module contains the ActivityManager class which is responsible for managing the activities and sessions of the users. The ActivityManager class is used to create, close and manage the activities and sessions of the users.

activity_book.manager.**_logger**

Logger object to log the messages.

Classes:

ActivityManager: Class to manage the activities and sessions of the users.

class activity_book.manager.**ActivityManager**

Bases: object

Description:

Class to manage the activities and sessions of the users.

__close_session()

Method to close a session.

create_session()

Method to create a session.

close_session()

Method to close a session.

create()

Method to create an activity.

`create_session`(*user*, *portal*, *domain*, *address*, *reason*)

Description:
create a session It tries to create a session and logs the activity.

Parameters

- `user` – User object.
- `portal` – Portal object.
- `domain` – Domain object.
- `address` – Address object.
- `reason` – Reason for creating the session.

Returns
Session object

Raises
`DatabaseError` – If there is an error while creating the session.

`close_session`(*username*, *portal*, *address*)

Description:
close a session

Parameters

- `username` – Username of the user.
- `portal` – Portal object.
- `address` – Address object.

Returns
None

Raises
`AcrowebException` – If there is an error while closing the session.

`create`(*user*, *portal*, *domain*, *address*, *app*, *text*)

Description:
create an activity. If there is no opened session, it creates a new session and then creates the activity. If there is more than one opened session, it closes the older ones and keeps the newest one.

Parameters

- `user` – User object.
- `portal` – Portal object.
- `domain` – Domain object.
- `address` – Address object.

- **app** – Application name.
- **text** – Activity text.

Returns
Activity object

Raises
AcrowebException – If there is an error while creating the activity.

2.4 activity_book.models module

Description:
This module contains the models for the activity_book app. The models are used to store the activities and sessions of the users.

Classes:
Session: Model to store the session details. Activity: Model to store the activity details.

class activity_book.models.**Session**(*id*, *address*, *username*, *portal*, *domain*, *date*, *closed*)

class activity_book.models.**Activity**(*id*, *session*, *date*, *application*, *activity*)

2.5 activity_book.serializers module

Description:
This module contains the serializers for the activity_book app. The serializers are used to serialize the data of the models.

Classes:
SessionSerializer: Serializer for the Session model. ActivitySerializer: Serializer for the Activity model.

class activity_book.serializers.**SessionSerializer**(**args*, ***kwargs*)

class activity_book.serializers.**ActivitySerializer**(**args*, ***kwargs*)

2.6 activity_book.views module

Description:
This module contains the views for the activity_book app. The views are used to handle the requests and responses of the activity_book app.

Classes:
ExternalActivityWriterView: Class to handle the external activity writer view. SessionListPagination: Class to handle the session list pagination. ActivityBookSessionView: Class to handle the activity book session view.

activity_book.views.**_logger**
Logger object to log the messages.

class activity_book.views.**ExternalActivityWriterView**(***kwargs*)
Bases: ViewSet

Description:
Class to handle the external activity writer view.

session()
Method to open/close a session.

create()
Method to create an activity.

permission_classes
List of permission classes.

permission_classes = [<class 'rest_framework.permissions.IsAuthenticated'>]

session(*request*)

Description:
open/close a session if the request method is post, it tries to create a session and logs the activity. else, it tries to close a session and logs the activity.

Parameters
request – Request object.

Returns
Response object.

Raises
AcrowebException – If an error occurs.

create(*request*)

Description:
create an activity It tries to create an activity and logs the activity.

Parameters
request – Request object.

Returns
Response object.

Raises
Response – If the request body is invalid.

class activity_book.views.**SessionListPagination**

Bases: PageNumberPagination

Description:
Class to handle the session list pagination.

page_size
Number of items per page.

page_size_query_param
Query parameter to get the page size.

max_page_size

Maximum number of items per page.

page_size = 10

page_size_query_param = 'page_size'

max_page_size = 20

class activity_book.views.**ActivityBookSessionView**(***kwargs*)

Bases: ReadOnlyModelViewSet

Description:
Class to handle the activity book session view.

permission_classes
List of permission classes.

serializer_class
Serializer class.

pagination_class
Pagination class.

get_queryset()
Method to get the query set.

retrieve()
Method to retrieve the activities of a session.

serializer_class
alias of *SessionSerializer*

pagination_class
alias of *SessionListPagination*

get_queryset()

Description:
get the query set It tries to get the sessions of the user.

Parameters
None

Returns
Session queryset filtered by domain and date range.

Raises
None –

retrieve(*request*, **args*, ***kwargs*)

Description:
retrieve the activities of a session It tries to get the activities of a session.

Parameters

- **request** – Request object.
- **args** – Arguments.
- **kwargs** – Keyword arguments.

Returns
Response object.

Raises
None –

CHAPTER

THREE

MANAGE MODULE

Django's command-line utility for administrative tasks.

`manage.main()`

Run administrative tasks.

CHAPTER

FOUR

INDICES AND TABLES

- genindex
- modindex
- search

4.1 Github Repository

PYTHON MODULE INDEX

a

acroweb3_portal.admin, 33
acroweb3_portal.admin_app, 32
acroweb3_portal.admin_app.idp_client, 3
acroweb3_portal.admin_app.idp_db, 6
acroweb3_portal.admin_app.idp_keycloak, 7
acroweb3_portal.admin_app.import_roles_in_db, 8
acroweb3_portal.admin_app.init_portal, 11
acroweb3_portal.admin_app.internal_views, 13
acroweb3_portal.admin_app.permissions, 14
acroweb3_portal.admin_app.serializers, 16
acroweb3_portal.admin_app.views, 19
acroweb3_portal.apps, 34
acroweb3_portal.asgi, 34
acroweb3_portal.auth, 35
acroweb3_portal.db_router, 39
acroweb3_portal.internal_views, 41
acroweb3_portal.models, 44
acroweb3_portal.serializers, 45
acroweb3_portal.settings, 46
acroweb3_portal.tests.commons, 32
acroweb3_portal.tests.dds_clients, 33
acroweb3_portal.tests.sentinel_client, 33
acroweb3_portal.urls, 47
acroweb3_portal.views, 47
acroweb3_portal.websocket, 54
acroweb3_portal.wsgi, 58
activity_book.admin, 59
activity_book.apps, 59
activity_book.manager, 59
activity_book.models, 61
activity_book.serializers, 61
activity_book.views, 61

m

manage, 65

INDEX

Symbols

__close_session() (*activity_book.manager.ActivityManager method*), 59
__notifiers (*acroweb3_portal.websocket.WSNotifier attribute*), 56
_logger (*in module acroweb3_portal.admin_app.idp_db*), 6
_logger (*in module activity_book.manager*), 59
_logger (*in module activity_book.views*), 61

A

acroweb3_portal.admin
    module, 33
acroweb3_portal.admin_app
    module, 32
acroweb3_portal.admin_app.idp_client
    module, 3
acroweb3_portal.admin_app.idp_db
    module, 6
acroweb3_portal.admin_app.idp_keycloak
    module, 7
acroweb3_portal.admin_app.import_roles_in_db
    module, 8
acroweb3_portal.admin_app.init_portal
    module, 11
acroweb3_portal.admin_app.internal_views
    module, 13
acroweb3_portal.admin_app.permissions
    module, 14
acroweb3_portal.admin_app.serializers
    module, 16
acroweb3_portal.admin_app.views
    module, 19
acroweb3_portal.apps
    module, 34
acroweb3_portal.asgi
    module, 34
acroweb3_portal.auth
    module, 35
acroweb3_portal.db_router
    module, 39
acroweb3_portal.internal_views
    module, 41
acroweb3_portal.models
    module, 44
acroweb3_portal.serializers
    module, 45
acroweb3_portal.settings
    module, 46
acroweb3_portal.tests.commons
    module, 32
acroweb3_portal.tests.dds_clients
    module, 33
acroweb3_portal.tests.sentinel_client
    module, 33
acroweb3_portal.urls
    module, 47
acroweb3_portal.views
    module, 47
acroweb3_portal.websocket
    module, 54
acroweb3_portal.wsgi
    module, 58
Acroweb3Admin (*class in acroweb3_portal.admin*), 33
Acroweb3Config (*class in acroweb3_portal.apps*), 34
Activity (*class in activity_book.models*), 61
activity_book.admin
    module, 59
activity_book.apps
    module, 59
activity_book.manager
    module, 59
activity_book.models
    module, 61
activity_book.serializers
    module, 61
activity_book.views
    module, 61
ActivityBookConfig (*class in activity_book.apps*), 59
ActivityBookSessionView (*class in activity_book.views*), 63
ActivityManager (*class in activity_book.manager*), 59
ActivityRouter (*class in acroweb3_portal.db_router*), 39
ActivitySerializer (*class in activity_book.serializers*), 61
add_notifier() (*acroweb3_portal.websocket.WSNotifier method*), 56
AdminApplicationsUpdateView (*class in acroweb3_portal.admin_app.views*), 29
AdminApplicationsView (*class in acroweb3_portal.admin_app.views*), 30
AdminConfigView (*class in acroweb3_portal.admin_app.views*), 20
AdminDomainGroupView (*class in acroweb3_portal.admin_app.views*), 30

AdminDomainsView (*class in acroweb3_portal.admin_app.views*), 27
AdminRolesView (*class in acroweb3_portal.admin_app.views*), 25
AdminUploadView (*class in acroweb3_portal.admin_app.views*), 24
AdminUserData (*class in acroweb3_portal.admin_app.idp_client*), 3
AdminUsersView (*class in acroweb3_portal.admin_app.views*), 20
allow_domain() (*acroweb3_portal.views.WidgetView method*), 53
allow_migrate() (*acroweb3_portal.db_router.ActivityRouter method*), 41
allow_relation() (*acroweb3_portal.db_router.ActivityRouter method*), 40
am_i_administrator() (*in module acroweb3_portal.admin_app.permissions*), 14
am_i_coordinator() (*in module acroweb3_portal.admin_app.permissions*), 15
am_i_cross_coordinator() (*in module acroweb3_portal.admin_app.permissions*), 15
app() (*acroweb3_portal.admin_app.views.AdminDomainsView method*), 28
Application (*class in acroweb3_portal.models*), 44
ApplicationAdmin (*class in acroweb3_portal.admin*), 33
ApplicationCategory (*class in acroweb3_portal.models*), 44
ApplicationCategorySerializer (*class in acroweb3_portal.serializers*), 46
ApplicationCategorySerializer.Meta (*class in acroweb3_portal.serializers*), 46
ApplicationCategoryView (*class in acroweb3_portal.views*), 48
ApplicationRole (*class in acroweb3_portal.models*), 44
ApplicationSerializer (*class in acroweb3_portal.serializers*), 46
ApplicationSerializer.Meta (*class in acroweb3_portal.serializers*), 46
ApplicationSerializerForList (*class in acroweb3_portal.admin_app.serializers*), 17
ApplicationSerializerForList.Meta (*class in acroweb3_portal.admin_app.serializers*), 17
ApplicationSerializerForWriter (*class in acroweb3_portal.admin_app.serializers*), 17
ApplicationSerializerForWriter.Meta (*class in acroweb3_portal.admin_app.serializers*), 17
auth_data (*acroweb3_portal.admin_app.idp_client.IDPClient attribute*), 5
AuthDomainSerializer (*class in acroweb3_portal.auth*), 35
AuthDomainSerializer.Meta (*class in acroweb3_portal.auth*), 35
authenticate() (*acroweb3_portal.auth.PortalAuthenticationReverseProxy method*), 39
AuthenticateView (*class in acroweb3_portal.internal_views*), 42
AuthenticationComponent (*class in acroweb3_portal.auth*), 35
avatar() (*acroweb3_portal.admin_app.views.AdminUsersView method*), 23

C

callback() (*acroweb3_portal.websocket.NewAcqWSNotifier method*), 58
callback() (*acroweb3_portal.websocket.SentinelWSNotifier method*), 58
cfg (*acroweb3_portal.admin_app.idp_client.IDPClient attribute*), 5
close_session() (*acroweb3_portal.internal_views.InternalActivityWrite method*), 43
close_session() (*activity_book.manager.ActivityManager method*), 59, 60
CommunityDataView (*class in acroweb3_portal.admin_app.internal_views*), 13
CommunityDomainGroupSerializer (*class in acroweb3_portal.admin_app.serializers*), 19
CommunityDomainGroupSerializer.Meta (*class in acroweb3_portal.admin_app.serializers*), 19
CommunityDomainSerializer (*class in acroweb3_portal.admin_app.serializers*), 19
CommunityDomainSerializer.Meta (*class in acroweb3_portal.admin_app.serializers*), 19
connect() (*acroweb3_portal.websocket.WSConsumer method*), 55
coordinator() (*acroweb3_portal.admin_app.views.AdminDomainsView method*), 29
create() (*acroweb3_portal.admin_app.serializers.ApplicationSerializerF method*), 17
create() (*acroweb3_portal.admin_app.serializers.DomainGroupSerialize method*), 19
create() (*acroweb3_portal.admin_app.serializers.DomainSerializerForW method*), 18
create() (*acroweb3_portal.admin_app.views.AdminRolesView method*), 26
create() (*acroweb3_portal.internal_views.AuthenticateView method*), 42
create() (*acroweb3_portal.internal_views.InternalActivityWriterView method*), 44
create() (*acroweb3_portal.internal_views.WSPushView method*), 43
create() (*activity_book.manager.ActivityManager method*), 59, 60
create() (*activity_book.views.ExternalActivityWriterView method*), 62
create_applications() (*in module acroweb3_portal.admin_app.init_portal*), 12
create_db_user() (*acroweb3_portal.admin_app.idp_client.AdminUserD method*), 4
create_domain() (*in module acroweb3_portal.admin_app.init_portal*), 12
create_portal_config() (*in module acroweb3_portal.admin_app.init_portal*), 11
create_session() (*activity_book.manager.ActivityManager method*), 59
create_user() (*acroweb3_portal.admin_app.idp_client.IDPClient method*), 5

create_user() (*acroweb3_portal.admin_app.idp_db.DBIDPClient method*), 6
create_user() (*acroweb3_portal.admin_app.idp_keycloak.KeycloakIDPClient method*), 8
create_user() (*in module acroweb3_portal.admin_app.init_portal*), 12
creationDate (*acroweb3_portal.admin_app.idp_client.AdminUserData attribute*), 4

D

DashboardSettings (*class in acroweb3_portal.models*), 45
DashboardSettingsView (*class in acroweb3_portal.views*), 51
db_for_read() (*acroweb3_portal.db_router.ActivityRouter method*), 40
db_for_write() (*acroweb3_portal.db_router.ActivityRouter method*), 40
db_user (*acroweb3_portal.admin_app.idp_client.AdminUserData attribute*), 4
DBIDPClient (*class in acroweb3_portal.admin_app.idp_db*), 6
default_auto_field (*acroweb3_portal.apps.Acroweb3Config attribute*), 34
default_auto_field (*activity_book.apps.ActivityBookConfig attribute*), 59
delete() (*acroweb3_portal.admin_app.serializers.ApplicationSerializerForWriter method*), 17
delete() (*acroweb3_portal.admin_app.views.AdminUsersView method*), 22
delete_user() (*acroweb3_portal.admin_app.idp_client.IDPClient method*), 5
delete_user() (*acroweb3_portal.admin_app.idp_db.DBIDPClient method*), 7
delete_user() (*acroweb3_portal.admin_app.idp_keycloak.KeycloakIDPClient method*), 8
destroy() (*acroweb3_portal.admin_app.views.AdminDomainsView method*), 27
destroy() (*acroweb3_portal.admin_app.views.AdminRolesView method*), 26
disconnect() (*acroweb3_portal.websocket.WSConsumer method*), 55
Domain (*class in acroweb3_portal.models*), 45
domain() (*acroweb3_portal.admin_app.views.AdminDomainGroupView method*), 32
domain() (*acroweb3_portal.admin_app.views.AdminUploadView method*), 25
domain() (*acroweb3_portal.views.ImageView method*), 50
DomainGroup (*class in acroweb3_portal.models*), 45
DomainGroupSerializer (*class in acroweb3_portal.admin_app.serializers*), 18
DomainGroupSerializer.Meta (*class in acroweb3_portal.admin_app.serializers*), 18
domains() (*acroweb3_portal.admin_app.internal_views.CommunityDataView method*), 13
domains() (*acroweb3_portal.admin_app.views.AdminDomainGroupView method*), 31
domains() (*acroweb3_portal.admin_app.views.AdminUsersView method*), 24
DomainSerializer (*class in acroweb3_portal.serializers*), 46
DomainSerializer.Meta (*class in acroweb3_portal.serializers*), 46
DomainSerializerForList (*class in acroweb3_portal.admin_app.serializers*), 17
DomainSerializerForList.Meta (*class in acroweb3_portal.admin_app.serializers*), 17
DomainSerializerForRetrieve (*class in acroweb3_portal.admin_app.serializers*), 18
DomainSerializerForRetrieve.Meta (*class in acroweb3_portal.admin_app.serializers*), 18
DomainSerializerForWriter (*class in acroweb3_portal.admin_app.serializers*), 18
DomainSerializerForWriter.Meta (*class in acroweb3_portal.admin_app.serializers*), 18
DomainWidgetSettings (*class in acroweb3_portal.models*), 45

E

email (*acroweb3_portal.admin_app.idp_client.AdminUserData attribute*), 4
ExternalActivityWriterView (*class in activity_book.views*), 61

F

fields (*acroweb3_portal.admin_app.serializers.ApplicationSerializerF attribute*), 18
fields (*acroweb3_portal.admin_app.serializers.ApplicationSerializerF attribute*), 17
fields (*acroweb3_portal.admin_app.serializers.CommunityDomainGr attribute*), 19
fields (*acroweb3_portal.admin_app.serializers.CommunityDomainSe attribute*), 19
fields (*acroweb3_portal.admin_app.serializers.DomainGroupSerializ attribute*), 19
fields (*acroweb3_portal.admin_app.serializers.DomainSerializerForI attribute*), 17
fields (*acroweb3_portal.admin_app.serializers.DomainSerializerForI attribute*), 18
fields (*acroweb3_portal.admin_app.serializers.DomainSerializerForV attribute*), 18
fields (*acroweb3_portal.auth.AuthDomainSerializer.Meta attribute*), 35
fields (*acroweb3_portal.serializers.ApplicationCategorySerializer.Me attribute*), 46
fields (*acroweb3_portal.serializers.ApplicationSerializer.Meta attribute*), 46
fields (*acroweb3_portal.serializers.DomainSerializer.Meta attribute*), 46
filter_horizontal (*acroweb3_portal.admin.UserDataAdmin attribute*), 34
firstName (*acroweb3_portal.admin_app.idp_client.AdminUserData attribute*), 4
formfield_overrides (*acroweb3_portal.admin.Acroweb3Admin attribute*), 33

G

get_auth_component() (*in module acroweb3_portal.auth*), 36
get_category() (*acroweb3_portal.admin_app.serializers.Application method*), 18

get_color() (*acroweb3_portal.admin_app.serializers.DomainSerializerForList method*), 17
get_color() (*acroweb3_portal.admin_app.serializers.DomainSerializerForRetrieve method*), 18
get_db_domain() (*in module acroweb3_portal.admin_app.import_roles_in_db*), 8
get_domain_img_url() (*in module acroweb3_portal.admin_app*), 32
get_enabled() (*acroweb3_portal.serializers.ApplicationSerializer method*), 46
get_group_img_url() (*in module acroweb3_portal.admin_app*), 32
get_icon_url() (*acroweb3_portal.admin_app.serializers.ApplicationSerializerForList method*), 18
get_icon_url() (*acroweb3_portal.serializers.ApplicationSerializer method*), 46
get_imgUrl() (*acroweb3_portal.admin_app.serializers.DomainGroupSerializer method*), 19
get_imgUrl() (*acroweb3_portal.admin_app.serializers.DomainSerializerForList method*), 17
get_imgUrl() (*acroweb3_portal.admin_app.serializers.DomainSerializerForRetrieve method*), 18
get_kc_client_id() (*in module acroweb3_portal.admin_app.import_roles_in_db*), 9
get_kc_user_id() (*in module acroweb3_portal.admin_app.import_roles_in_db*), 9
get_portal_and_issuer_by_host() (*in module acroweb3_portal.auth*), 37
get_portal_config_by_host() (*in module acroweb3_portal.auth*), 36
get_queryset() (*acroweb3_portal.admin_app.views.AdminDomainGroupView method*), 31
get_queryset() (*acroweb3_portal.admin_app.views.AdminDomainsView method*), 27
get_queryset() (*acroweb3_portal.views.ApplicationCategoryView method*), 49
get_queryset() (*activity_book.views.ActivityBookSessionView method*), 63
get_request_db_user_data() (*in module acroweb3_portal.admin_app*), 32
get_roles() (*acroweb3_portal.serializers.ApplicationSerializer method*), 46
get_serializer_class() (*acroweb3_portal.admin_app.views.AdminDomainsView method*), 28
get_settings() (*acroweb3_portal.views.UserAppSettingsView method*), 54
get_user_img_url() (*in module acroweb3_portal.admin_app*), 32
getIDPClient() (*in module acroweb3_portal.admin_app.idp_client*), 5
group() (*acroweb3_portal.admin_app.views.AdminUploadView method*), 25
group() (*acroweb3_portal.views.ImageView method*), 50
groups() (*acroweb3_portal.admin_app.internal_views.CommunityDataView method*), 14

H

has_permission() (*acroweb3_portal.admin_app.permissions.IsAdministratorPermission method*), 16
has_permission() (*acroweb3_portal.admin_app.permissions.IsCoordinatorPermission method*), 15

I

icon_svg_view() (*acroweb3_portal.admin.ApplicationAdmin method*), 34
idp_clients (*in module acroweb3_portal.admin_app.idp_client*), 3
IDPClient (*class in acroweb3_portal.admin_app.idp_client*), 4
image() (*acroweb3_portal.admin_app.views.AdminDomainGroupView method*), 31
image() (*acroweb3_portal.admin_app.views.AdminDomainsView method*), 28
imageView (*class in acroweb3_portal.views*), 49
img_view() (*acroweb3_portal.admin.UserDataAdmin method*), 34
init_db() (*in module acroweb3_portal.tests.commons*), 32
InternalActivityWriterView (*class in acroweb3_portal.internal_views*), 43
InternalDomainView (*class in acroweb3_portal.internal_views*), 42
IsAdministratorPermission (*class in acroweb3_portal.admin_app.permissions*), 16
IsCoordinatorPermission (*class in acroweb3_portal.admin_app.permissions*), 15
issuer (*acroweb3_portal.admin_app.idp_client.IDPClient attribute*), 5
IssuerView (*class in acroweb3_portal.internal_views*), 41

K

KafkaWSNotifier (*class in acroweb3_portal.websocket*), 57
kc_client_id_cache (*in module acroweb3_portal.admin_app.import_roles_in_db*), 8
kc_user_id_cache (*in module acroweb3_portal.admin_app.import_roles_in_db*), 8
KeycloakIDPClient (*class in acroweb3_portal.admin_app.idp_keycloak*), 8

L

lastName (*acroweb3_portal.admin_app.idp_client.AdminUserData attribute*), 4
list() (*acroweb3_portal.admin_app.internal_views.CommunityDataView method*), 13
list() (*acroweb3_portal.admin_app.views.AdminConfigView method*), 20
list() (*acroweb3_portal.admin_app.views.AdminRolesView method*), 26
list() (*acroweb3_portal.admin_app.views.AdminUsersView method*), 20
list() (*acroweb3_portal.internal_views.IssuerView method*), 42
list() (*acroweb3_portal.views.DashboardSettingsView method*), 51
list() (*acroweb3_portal.views.PortalConfigView method*), 51
list() (*acroweb3_portal.views.ServerTimeView method*), 48
list() (*acroweb3_portal.views.UserImagesView method*), 48
list() (*acroweb3_portal.views.WidgetView method*), 52

list_display (*acroweb3_portal.admin.ApplicationAdmin attribute*), 33
list_display (*acroweb3_portal.admin.UserDataAdmin attribute*), 34
list_display_links (*acroweb3_portal.admin.ApplicationAdmin attribute*), 33
list_display_links (*acroweb3_portal.admin.UserDataAdmin attribute*), 34
list_filter (*acroweb3_portal.admin.ApplicationAdmin attribute*), 33
list_filter (*acroweb3_portal.admin.UserDataAdmin attribute*), 34
list_max_show_all (*acroweb3_portal.admin.Acroweb3Admin attribute*), 33
list_per_page (*acroweb3_portal.admin.Acroweb3Admin attribute*), 33

M

main() (*in module manage*), 65
manage
    module, 65
manage_portals() (*in module acroweb3_portal.admin_app.import_roles_in_db*), 10
match() (*acroweb3_portal.websocket.WSConsumer method*), 55
max_page_size (*activity_book.views.SessionListPagination attribute*), 62, 63
me() (*acroweb3_portal.admin_app.views.AdminUsersView method*), 21
media (*acroweb3_portal.admin.Acroweb3Admin property*), 33
media (*acroweb3_portal.admin.ApplicationAdmin property*), 34
media (*acroweb3_portal.admin.UserDataAdmin property*), 34
model (*acroweb3_portal.admin_app.serializers.ApplicationSerializerForListMeta attribute*), 17
model (*acroweb3_portal.admin_app.serializers.ApplicationSerializerForWriterMeta attribute*), 17
model (*acroweb3_portal.admin_app.serializers.DomainGroupSerializer.Meta attribute*), 18
model (*acroweb3_portal.admin_app.serializers.DomainSerializerForListMeta attribute*), 17
model (*acroweb3_portal.admin_app.serializers.DomainSerializerForRetrieve.Meta attribute*), 18
model (*acroweb3_portal.admin_app.serializers.DomainSerializerForWriter.Meta attribute*), 18
model (*acroweb3_portal.auth.AuthDomainSerializer.Meta attribute*), 35
model (*acroweb3_portal.serializers.ApplicationCategorySerializer.Meta attribute*), 46
model (*acroweb3_portal.serializers.ApplicationSerializer.Meta attribute*), 46
model (*acroweb3_portal.serializers.DomainSerializer.Meta attribute*), 46
module
    acroweb3_portal.admin, 33
    acroweb3_portal.admin_app, 32
    acroweb3_portal.admin_app.idp_client, 3
    acroweb3_portal.admin_app.idp_db, 6
    acroweb3_portal.admin_app.idp_keycloak, 7
    acroweb3_portal.admin_app.import_roles_in_db, 8
    acroweb3_portal.admin_app.init_portal, 11
    acroweb3_portal.admin_app.internal_views, 13
    acroweb3_portal.admin_app.permissions, 14
    acroweb3_portal.admin_app.serializers, 16
    acroweb3_portal.admin_app.views, 19
    acroweb3_portal.apps, 34
    acroweb3_portal.asgi, 34
    acroweb3_portal.auth, 35
    acroweb3_portal.db_router, 39
    acroweb3_portal.internal_views, 41
    acroweb3_portal.models, 44
    acroweb3_portal.serializers, 45
    acroweb3_portal.settings, 46
    acroweb3_portal.tests.commons, 32
    acroweb3_portal.tests.dds_clients, 33
    acroweb3_portal.tests.sentinel_client, 33
    acroweb3_portal.urls, 47
    acroweb3_portal.views, 47
    acroweb3_portal.websocket, 54
    acroweb3_portal.wsgi, 58
    activity_book.admin, 59
    activity_book.apps, 59
    activity_book.manager, 59
    activity_book.models, 61
    activity_book.serializers, 61
    activity_book.views, 61
    manage, 65
mydomains() (*acroweb3_portal.views.WidgetView method*), 52

N

NewAcquisitionNotifier (*class in acroweb3_portal.websocket*), 58
notifier_listener (*acroweb3_portal.websocket.WSConsumer attribute*), 55
notify() (*acroweb3_portal.websocket.WSConsumer static method*), 56
notify() (*acroweb3_portal.websocket.WSNotifier method*), 57
notify_portal() (*acroweb3_portal.websocket.WSConsumer static method*), 56

O

on_message() (*acroweb3_portal.websocket.KafkaWSNotifier method*), 57
open_session() (*acroweb3_portal.internal_views.InternalActivityWri method*), 43

P

page_size (*activity_book.views.SessionListPagination attribute*), 62, 63
page_size_query_param (*activity_book.views.SessionListPagination attribute*), 62, 63
pagination_class (*activity_book.views.ActivityBookSessionView attribute*), 63
parser_classes (*acroweb3_portal.admin_app.views.AdminUploadVi attribute*), 24

permission_classes (*acroweb3_portal.admin_app.views.AdminApplicationsSpecificView attribute*), 30
permission_classes (*acroweb3_portal.admin_app.views.AdminApplicationsView attribute*), 30
permission_classes (*acroweb3_portal.admin_app.views.AdminConfigView attribute*), 20
permission_classes (*acroweb3_portal.admin_app.views.AdminDomainGroupsView attribute*), 31
permission_classes (*acroweb3_portal.admin_app.views.AdminDomainsView attribute*), 27
permission_classes (*acroweb3_portal.admin_app.views.AdminRolesView attribute*), 26
permission_classes (*acroweb3_portal.admin_app.views.AdminUploadView attribute*), 24
permission_classes (*acroweb3_portal.admin_app.views.AdminUsersView attribute*), 20
permission_classes (*acroweb3_portal.views.ApplicationCategoryView attribute*), 49
permission_classes (*acroweb3_portal.views.DashboardSettingsView attribute*), 51
permission_classes (*acroweb3_portal.views.ImageView attribute*), 49
permission_classes (*acroweb3_portal.views.UserAppSettingsView attribute*), 54
permission_classes (*acroweb3_portal.views.UserImagesView attribute*), 48
permission_classes (*acroweb3_portal.views.WidgetView attribute*), 52
permission_classes (*activity_book.views.ActivityBookSessionView attribute*), 63
permission_classes (*activity_book.views.ExternalActivityWriterView attribute*), 62
phone (*acroweb3_portal.admin_app.idp_client.AdminUserData attribute*), 4
PortalAuthentication (*class in acroweb3_portal.auth*), 37
PortalAuthenticationReverseProxy (*class in acroweb3_portal.auth*), 38
PortalConfig (*class in acroweb3_portal.models*), 44
PortalConfigView (*class in acroweb3_portal.views*), 51
props() (*acroweb3_portal.admin_app.views.AdminUsersView method*), 22

Q

queryset (*acroweb3_portal.admin_app.views.AdminApplicationsSpecificView attribute*), 30
queryset (*acroweb3_portal.admin_app.views.AdminApplicationsView attribute*), 30
queryset (*acroweb3_portal.internal_views.InternalDomainView attribute*), 42

R

read_data_file() (*in module acroweb3_portal.admin_app.init_portal*), 11
readonly_fields (*acroweb3_portal.admin.ApplicationAdmin attribute*), 34
readonly_fields (*acroweb3_portal.admin.UserDataAdmin attribute*), 34
reactivate_notifier() (*acroweb3_portal.websocket.WSNotifier method*), 57
reset_user_password() (*acroweb3_portal.admin_app.idp_client.IDPC method*), 5
reset_user_password() (*acroweb3_portal.admin_app.idp_db.DBIDPC method*), 7
reset_user_password() (*acroweb3_portal.admin_app.idp_keycloak.Key method*), 8
resetpassword() (*acroweb3_portal.admin_app.views.AdminUsersView method*), 22
retrieve() (*activity_book.views.ActivityBookSessionView method*), 63
roles() (*acroweb3_portal.admin_app.views.AdminUsersView method*), 23
roles() (*acroweb3_portal.admin_app.views.AdminUsersView method*), 23
route_app_labels (*acroweb3_portal.db_router.ActivityRouter attribute*), 40

S

save() (*acroweb3_portal.admin_app.views.AdminUsersView method*), 21
save() (*acroweb3_portal.models.UserData method*), 45
save_as (*acroweb3_portal.admin.Acroweb3Admin attribute*), 33
save_widget_settings() (*acroweb3_portal.views.WidgetView method*), 52
search_fields (*acroweb3_portal.admin.Acroweb3Admin attribute*), 33
search_fields (*acroweb3_portal.admin.ApplicationAdmin attribute*), 33
search_fields (*acroweb3_portal.admin.UserDataAdmin attribute*), 34
SentinelWSNotifier (*class in acroweb3_portal.websocket*), 58
serializer_class (*acroweb3_portal.admin_app.views.AdminApplicatio attribute*), 30
serializer_class (*acroweb3_portal.admin_app.views.AdminApplicatio attribute*), 30
serializer_class (*acroweb3_portal.admin_app.views.AdminDomainGr attribute*), 31
serializer_class (*acroweb3_portal.internal_views.InternalDomainVie attribute*), 42
serializer_class (*acroweb3_portal.views.ApplicationCategoryView attribute*), 49
serializer_class (*activity_book.views.ActivityBookSessionView attribute*), 63
ServerTimeView (*class in acroweb3_portal.views*), 47
Session (*class in activity_book.models*), 61
session() (*activity_book.views.ExternalActivityWriterView method*), 62
SessionListPagination (*class in activity_book.views*), 62
SessionSerializer (*class in activity_book.serializers*), 61
set_from_db() (*acroweb3_portal.admin_app.idp_client.AdminUserData method*), 4

set_from_props() (*acroweb3_portal.admin_app.idp_client.AdminUserData method*), 4
set_users_domains() (*in module acroweb3_portal.admin_app.import_roles_in_db*), 10
set_users_roles() (*in module acroweb3_portal.admin_app.import_roles_in_db*), 10

T

to_dict() (*acroweb3_portal.admin_app.idp_client.AdminUserData method*), 4
translate() (*acroweb3_portal.views.TranslationView method*), 53
TranslationView (*class in acroweb3_portal.views*), 53

U

update() (*acroweb3_portal.admin_app.serializers.ApplicationSerializerForWriter method*), 17
update_db_user() (*acroweb3_portal.admin_app.idp_client.AdminUserData method*), 4
user() (*acroweb3_portal.admin_app.views.AdminUploadView method*), 24
user() (*acroweb3_portal.admin_app.views.AdminUsersView method*), 21
user() (*acroweb3_portal.views.ImageView method*), 49
user_domains() (*acroweb3_portal.admin_app.internal_views.CommunityDataView method*), 14
user_exists() (*acroweb3_portal.admin_app.idp_client.IDPClient method*), 5
user_exists() (*acroweb3_portal.admin_app.idp_db.DBIDPClient method*), 6
user_exists() (*acroweb3_portal.admin_app.idp_keycloak.KeycloakIDPClient method*), 8
UserAppSettings (*class in acroweb3_portal.models*), 45
UserAppSettingsView (*class in acroweb3_portal.views*), 54
UserData (*class in acroweb3_portal.models*), 45
UserDataAdmin (*class in acroweb3_portal.admin*), 34
UserImagesView (*class in acroweb3_portal.views*), 48
username (*acroweb3_portal.admin_app.idp_client.AdminUserData attribute*), 4
users() (*acroweb3_portal.admin_app.views.AdminDomainsView method*), 29
UserWidgetSettings (*class in acroweb3_portal.models*), 45

V

validate_token() (*acroweb3_portal.auth.AuthenticationComponent method*), 35
validate_token() (*acroweb3_portal.auth.PortalAuthentication method*), 37
validate_token() (*acroweb3_portal.auth.PortalAuthenticationReverseProxy method*), 38
validate_token_and_build_auth_data() (*acroweb3_portal.auth.AuthenticationComponent method*), 36
validate_token_and_build_auth_data() (*acroweb3_portal.auth.PortalAuthentication method*), 38
validate_token_and_build_auth_data() (*acroweb3_portal.auth.PortalAuthenticationReverseProxy method*), 39
validate_unique() (*acroweb3_portal.models.UserData method*), 45

W

Widget (*class in acroweb3_portal.models*), 45
WidgetInstance (*class in acroweb3_portal.models*), 45
WidgetType (*class in acroweb3_portal.models*), 45
WidgetView (*class in acroweb3_portal.views*), 52
WSConsumer (*class in acroweb3_portal.websocket*), 54
WSNotifier (*class in acroweb3_portal.websocket*), 56
WSPushView (*class in acroweb3_portal.internal_views*), 42
WSTokenAuthMiddleware (*class in acroweb3_portal.websocket*), 54